

Code Explanation

E2E AC TXDBH DP YUYU Creation Workflow Configuration

The provided Python code is used to configure the E2E AC TXDBH DP YUYU Creation workflow. It defines a data class `WorkflowConfig` to hold the configuration parameters and a function `get\_config` to parse command line arguments and create an instance of `WorkflowConfig`.

Overview of the Code

The code is designed to parse command line arguments and environment variables to create a workflow configuration. It uses the `argparse` library to parse command line arguments and the `os` library to access environment variables. The configuration is stored in a `WorkflowConfig` data class instance.

Step 1: Define the Workflow Configuration Data Class

The code defines a data class `WorkflowConfig` to hold the configuration parameters for the E2E AC TXDBH DP YUYU Creation workflow. This data class includes attributes for database connections, schema and table names, file paths, and process parameters.

```
@dataclass
class WorkflowConfig:
    """Configuration for the E2E AC TXDBH DP YUYU Creation workflow.
    """
    # Database connections
    db_connection_stg: str
    db_connection_ods: str

    # Schema and table names
    stg_schema: str
    stg_table: str
    lkp_schema: str
    lkp_tbl_1: str # T_YUYU_CLNT
    lkp_tbl_2: str # T_YUYUK_CLN

    # File paths
    output_dir: str
    output_file_yuyu: str
    output_file_cnt_chk: str
    bad_file_yuyu: str
    bad_file_cnt_chk: str
    trigger_file: str

    # Process parameters
    process_userid: str
    file_timestamp: str
```

Step 2: Parse Command Line Arguments

The `get\_config` function uses the `argparse` library to parse command line arguments. It defines a parser with a description of the workflow and adds arguments for the required configuration parameters.

```
def get_config():
    """Parse command line arguments and create workflow configuration.
    """
```

```

    parser = argparse.ArgumentParser(description="E2E AC TXDBH DP YU
YU Creation Workflow")

    # Database connection parameters (use environment variables or D
atabricks secrets)
    parser.add_argument("--stg_schema", required=True, help="Schema
for staging table")
    parser.add_argument("--stg_table", required=True, help="Staging
table name")
    parser.add_argument("--lkp_schema", required=True, help="Schema
for lookup tables")
    parser.add_argument("--lkp_tbl_1", required=True, help="First lo
okup table (T_YUYU_CLNT)")
    parser.add_argument("--lkp_tbl_2", required=True, help="Second l
ookup table (T_YUYUK_CLN)")

    # File parameters
    parser.add_argument("--output_dir", required=True, help="Output
directory for files")
    parser.add_argument("--output_file_yuyu", required=True, help="O
utput filename for YUYU file")
    parser.add_argument("--trigger_file", required=True, help="Trigg
er filename")
    parser.add_argument("--process_userid", required=True, help="Pro
cess user ID for audit")

    args = parser.parse_args()

```

Step 3: Generate Timestamp and Get Database Connections

The code generates a timestamp for filenames using the current date and time. It also retrieves database connections from environment variables.

```

# Generate timestamp for filenames
file_timestamp = datetime.now().strftime("%Y%m%d%H%M%S")

# Get database connections from environment or Databricks secrets
db_connection_stg = os.environ.get("DBConnection_E2E_ORA_STG")
db_connection_ods = os.environ.get("DBConnection_E2E_ORA_ODS")

```

Step 4: Construct Derived Filenames

The code constructs derived filenames for count check, bad YUYU, and bad count check files using the generated timestamp.

```

# Construct derived filenames
output_file_cnt_chk = f"SourceCount_{file_timestamp}.csv"
bad_file_yuyu = f"Bad_YUYU_{file_timestamp}.txt"
bad_file_cnt_chk = f"Bad CountCheck {file_timestamp}.txt"

```

Step 5: Create Workflow Configuration Instance

The code creates an instance of `WorkflowConfig` using the parsed command line arguments, generated timestamp, and retrieved database connections.

```
return WorkflowConfig(  
    db_connection_stg=db_connection_stg,  
    db_connection_ods=db_connection_ods,  
    stg_schema=args.stg_schema,  
    stg_table=args.stg_table,  
    lkp_schema=args.lkp_schema,  
    lkp_tbl_1=args.lkp_tbl_1,  
    lkp_tbl_2=args.lkp_tbl_2,  
    output_dir=args.output_dir,  
    output_file_yuyu=args.output_file_yuyu,  
    output_file_cnt_chk=output_file_cnt_chk,  
    bad_file_yuyu=bad_file_yuyu,  
    bad_file_cnt_chk=bad_file_cnt_chk,  
    trigger_file=args.trigger_file,  
    process_userid=args.process_userid,  
    file_timestamp=file_timestamp  
)
```